Problem 2

In [68]:
```
import qutip as qt
from qutip import Qobj
import numpy as np

# Define polarization states
H = Qobj([[1], [0]])  # Horizontal polarization
A = (1 / np.sqrt(2)) * Qobj([[1], [-1]])  # Anti-diagonal polarization
V = Qobj([[0], [1]])  # Vertical polarization
D = (1 / np.sqrt(2)) * Qobj([[1], [1]])  # Diagonal polarization

num_send = 10000  # the number of photons Alice send
```

In [69]:
```
def ADC(qobj):
    '''
    translate qobj to char variable, so that the public channel function is

    Parameter(s):
        qobj:the qobj needed to be translated

    Return(s):
        the corresponding character to the input.
    '''
    if qobj==H:
        return "H"
    if qobj==V:
        return "V"
    if qobj==D:
        return "D"
    if qobj==A:
        return "A"
```

The following box belongs to Alice

In [70]:
```
def Alice(num_photon):
    '''
    “Alice” is a function that accepts an integer num_photon which is the nu

    Parameter(s):
        num_photon: the number of photon

    Return(s):
        a list that has two columns, the first column is the index(starting
    '''
    p = 0.2
    p1 = 0.4
    return [(i+1, np.random.choice([H, A]), np.random.choice([1, 2], p=[1-p,
```

The following box belongs to Bob

In [71]:
```
def Bob(photon_list):
    '''
    “Bob” accepts the output of “Alice” and returns only the events where he

    Parameter(s):
        photon_list: the output of Alice, a list that consist of many tuple

    Return(s):
        received_message: the output of Bob, photons that Bob get a pin and
    '''
    received_message=[]
    for index, photon, _ in photon_list:
        state = 'A' if photon == A else 'H'
        receiver = np.random.choice(['V','D'])
        if receiver == 'V' and state == 'H' or receiver == 'D' and state =='
            pass  # No pin Possible
        elif receiver == 'D' and state == 'H':
            pin = np.random.choice(['H', 'Null'], p=[0.1, 0.9])  # simulate
            if pin != 'Null':
                received_message.append((index, pin))  # register the pin
        elif receiver == 'V' and state == 'A':  # similar to the last elif s
            pin = np.random.choice(['A', 'Null'], p=[0.1, 0.9])
            if pin != 'Null':
                received_message.append((index, pin))
    return received_message
```

In [72]:
```
def Bob_1(photon_list):
    '''
    “Bob” accepts the output of “Alice” and returns only the events where he

    Parameter(s):
        photon_list: the output of Alice, a list that consist of many tuple

    Return(s):
        received_message: the output of Bob, photons that Bob get a pin and
    '''
    received_message=[]
    for index, photon, _ in photon_list:
        state = 'A' if photon == A else 'H'
        receiver = np.random.choice(['V','D'])
        if receiver == 'V' and state == 'H' or receiver == 'D' and state =='
            pass  # No pin Possible
        elif receiver == 'D' and state == 'H':
            pin = np.random.choice(['H', 'Null'])  # simulate the process of
            if pin != 'Null':
                received_message.append((index, pin))  # register the pin
        elif receiver == 'V' and state == 'A':  # similar to the last elif s
            pin = np.random.choice(['A', 'Null'])
            if pin != 'Null':
                received_message.append((index, pin))
    return received_message
```

Eve

In [73]:
```
def Eve(photon_list):
    '''
    a spy “Eve” who takes the output that “Alice” produces, performs the sam

    Parameter(s):
        photon_list: the output of Eve, a list that consist of many tuple of

    Return(s):
        received_message: the output of Bob, photons that Bob get a pin and
    '''
    received_message=[]
    for index, photon, num_photon in photon_list:
        state = 'A' if photon == A else 'H'
        receiver = np.random.choice(['V','D'])
        if num_photon != 1:
            if receiver == 'V' and state == 'H' or receiver == 'D' and state
                received_message.append((index, receiver, 2))  # indecate th
            elif receiver == 'D' and state == 'H':
                pin = np.random.choice(['H', receiver])  # simulate the proc
                received_message.append((index, pin, 2))
            elif receiver == 'V' and state == 'A':  # similar to the last el
                pin = np.random.choice(['A', receiver])
                received_message.append((index, pin, 2))
        else:
            # received_message.append((index, state, 1))
            pass

        forged_message = []

    for index, click, num_p in received_message:
        if num_p != 1:
            if click == "V":
                forged_message.append((index, H, num_p))
            elif click == "D":
                forged_message.append((index, A, num_p))
            elif click == "H":
                forged_message.append((index, H, num_p))
            elif click == "A":
                forged_message.append((index, A, num_p))
        else:
            # forged_message.append((index, click, num_p))
            pass
    while len(forged_message) > 0.1 * num_send:
        del forged_message[np.random.randint(0, len(forged_message))]
    return forged_message
```

In [74]:
```
def public_channel(click_received, photon_sent):
    '''
    This channel is used to varify the click Bob got with the photon Alice s

    Parameter(s):
        click_received: a part of the click bob received
```

```
    Return(s):
        check_list: a list that contains for each click Bob claim he receive
        true_rate: the proportion of clicks that Bob got it right.
    '''
    check_list = []
    for i, t in enumerate(click_received) :
        index, click = t
        if ADC(photon_sent[index - 1][1]) == click:
            check_list.append((index, True))
        else:
            check_list.append((index, False))
    # calculate the ratio of correct count to the entire compared clicks
    true_count = 0
    for i, b in check_list:
        if b:
            true_count += 1
    true_rate = true_count/len(check_list)
    error_rate = 1-true_rate
    first_half_count = sum(1 for i, _ in check_list if 1 <= i <= num_send /
    second_half_count = sum(1 for i, _ in check_list if num_send / 2 < i <=
    return check_list, error_rate, first_half_count, second_half_count
```

Without Eve

In [99]:
```
photon_sent = Alice(num_send)  # the photons Alice sent
click_received = Bob(photon_sent[0:len(photon_sent)])  # the click Bob got,
temp, rate, o1, o2 = public_channel(click_received, photon_sent)
print("Error rate: ", rate)
print("The count in the first and second half",o1, o2)
```

```
Error rate:  0.0
The count in the first and second half 237 242
```

With Eve

In [100…
```
rate_list = []
num_cr = 0
repeat = 1
for i in range(repeat):
    photon_sent = Alice(num_send)  # the photons Alice sent
    photon_sent_E = Eve(photon_sent)
    click_received = Bob_1(photon_sent_E)  # the click Bob got
    num_cr += len(click_received)
    temp, rate, ftr, ltr = public_channel(click_received, photon_sent)
    rate_list.append(rate)
num_cr = num_cr/repeat
print('Error rate: ',np.mean(rate_list), f'click/photon sent:{num_cr/num_ser
print("The count in the first and second half of the sent photon",ftr, ltr)
```

```
Error rate:  0.22857142857142854 click/photon sent:0.0245
The count in the first and second half of the sent photon 84 161
```